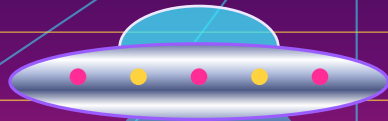


S O I S S A N S T E M P S

STRUDEL × HYDRA

live coding musical, visuel, collectif



CONTENU

PARTIE I · PREMIERS PAS

démarrer · cycle · mini-notation · batterie et euclide · empiler et muter

PARTIE II · MATIÈRE SONORE

notes et gammes · accords et arpèges · effets · enveloppes · signaux · transformer · samples et breaks

PARTIE III · CONSTRUIRE UN MORCEAU

sept écritures · faire entrer · arrange · pick · les gestes

PARTIE IV · VISUEL

visus · hydra solo · hydra dans flock

PARTIE V · COLLECTIF

strudel.cc contre flock · conventions de session · arrangement par mutes

PARTIE VI · EXEMPLES

techno · deep house · acid · jungle · boom bap · trap · ambient · synthwave · polyrythmie
jazz · bossa · valse · baroque · reggae · percussions indiennes · bluegrass · gamelan · fanfare

PARTIE VII · RÉFÉRENCES

antisèche · dépannage · plus loin · sources

DEUX RACCOURCIS UTILES

Ctrl + Entrée

actualiser

Ctrl + .

couper

À RETENIR

Une erreur n'arrête pas le son : il faut corriger puis réactualiser.

SUR FLOK

Alt + Entrée actualise tous les panneaux. **Ctrl + .** ne coupe que le panneau courant.

PARTIE I

PREMIERS PAS

cycle • mini-notation • batterie • empilement



DÉMARRER EN SOLO

1. Ouvrir strudel.cc, un exemple est déjà chargé.
2. `Ctrl+Entrée` ou le bouton lecture : ça joue.
3. `Ctrl+.` : silence.
4. Effacer, écrire, actualiser.

```
// 4 sons sur un cycle, en boucle  
sound("bd hh sd hh")
```

Aucune installation, sauvegarde, ou export nécessaire. Le texte dans le moteur en ligne **suffit**.

DÉMARRER À PLUSIEURS

1. Une personne ouvre `flok.cc` puis **Create session**.
2. Menu **Command**, puis **Configure** : les cibles séparées par des virgules, 8 panneaux maximum.
3. Chacun saisit un pseudo, l'URL sert d'invitation.
4. Un panneau = une personne, ou un rôle.

```
strudel, strudel, strudel, hydra
```

Ctrl/Cmd + Entrée

bloc ou sélection

Alt + Entrée

tout le panneau

Shift + Entrée

la ligne

Ctrl/Alt + .

couper ce panneau

Ctrl/Cmd + Shift + H

masquer les panneaux

Raccourcis : documentation flok.

⚠

La page est publique : toute personne disposant de l'URL exécute du code dans le navigateur des autres
→ Ne pas diffuser l'adresse pendant la session.

NOTION DE CYCLE

Un cycle est une boucle de durée fixe.

Une séquence "a b c d" = cycle divisé en parts égales, une par élément.

```
sound("bd")           // 1 par cycle  
sound("bd sd")         // 2 par cycle  
sound("bd sd hh oh")   // 4 par cycle
```

- ▶ 1 événement par cycle
- ▶ 2 événements, une demi-boucle chacun
- ▶ 4 événements

Ajouter des éléments **densifie** le cycle, ça ne le rallonge pas !

(« *the longer the sequence, the faster it runs* », workshop Strudel officiel)

CLÉ

Rien ne se place sur une timeline. Tout se décrit comme un remplissage de cycle. ■

LE TEMPO

Strudel compte en **cycles par minute** (cpm), pas en battements (BPM). Un cycle ne contient aucun temps fixe : la mini-notation le découpe.

BPM = cpm × nombre de temps écrits dans le cycle. Défaut : 30 cpm (`setcps(.5)`, 1 cycle = 2 s).

```
setcpm(120/4)           // 30 cpm : 1 cycle = 2 s
sound("bd hh sd hh")    // 4 sons = 4 temps : 120 BPM
sound("bd*8")           // même cpm, 8 kicks par cycle : 240 coups par minute
```

Défaut 0,5 cps et équivalence avec 120 BPM : page « Cycles » de la doc Strudel.

CONSÉQUENCE UTILE

`setcpm(186/3)` + motif en 3 éléments ("bd hh hh") = valse à 186 BPM. `setcpm(160/7)` + 7 éléments = 7/8.
→ La métrique vient du nombre d'éléments écrits ; le diviseur règle le tempo pour qu'un élément dure un temps.

/?\

En session collective sur flok, une seule personne écrit `setcpm()`.

MINI-NOTATION : LES QUATORZE SIGNES

espace	"bd sd"	séquence	!	"bd!3 sd"	répète
~ -	"bd ~ sd"	silence	@	"bd@3 sd"	allonge
*	"hh*8"	accélère	?	"[hh?]*8"	probabilité 50 %
/	"bd/2"	ralentit	:	"hh:3"	numéro d'échantillon
[]	"bd [sd cp]"	sous-séquence	(,)	"bd(3,8)"	euclidien
< >	"<bd sd>"	un par cycle		"bd cp"	tirage par cycle
,	"bd*2, hh*4"	superposition	{ , }	"{bd sd, hh hh hh}"	polymètre

ACCÉLÈRE, RALENTIT, RÉPÈTE, ALLONGE

```
sound("bd*2 sd") // [bd bd] sd
sound("bd!2 sd") // bd bd sd
sound("bd@2 sd") // bd tient 2/3
sound("bd/2 sd") // bd 1 cycle/2
```

<code>*n</code>	accélère	joue l'élément n fois dans son pas . Nombre de pas inchangé.
<code>/n</code>	ralentit	étale l'élément sur n cycles : il ne sonne qu'un cycle sur n.
<code>!n</code>	répète	recopie l'élément n fois, chaque copie prend un pas .
<code>@n</code>	allonge	l'élément pèse n pas au lieu d'un, les autres raccourcissent.

MÊME RÉSULTAT

Seul dans la chaîne : `"bd*3"` = `"bd!3"` = `"bd bd bd"`.

`"bd@3 sd"` et `"bd ~ ~ sd"` : mêmes attaques, seule la durée de `bd` change (audible sur une note tenue, pas sur un coup de batterie).

`"<bd sd>"` = `"[bd sd]/2"` : l'alternance est une séquence ralentie. ■

DIFFÉRENT

Dès qu'un autre élément suit : `"bd*2 sd"` garde 2 pas, `"bd!2 sd"` en a 3. Le `sd` n'arrive pas au même moment. ■

MINI-NOTATION : TOUT SE COMBINE

```
// [ ] groupe, ~ silence, < > alterne, ? hasard, *2 double  
sound("bd [~ sd] <hh oh> [cp cp?]*2")
```

Ces signes s'imbriquent sans limite de profondeur. Un pattern complexe reste une seule chaîne de caractères.

DÉBOGAGE

Pendant la lecture, la mini-notation s'allume au rythme des événements. Un caractère qui ne clignote jamais ne sert à rien. Les panneaux Hydra de flok n'ont pas ce surlignage.

MANIP

Un beat sans virgule : uniquement `bd sd hh oh`, `[ ]`, `~` et `*`. Puis un `?` sur un seul élément.

LA BATTERIE

Un nom = un **échantillon** : court fichier son enregistré (**bd** = un coup de grosse caisse). Une **banque** = le kit d'une boîte à rythmes : mêmes noms, autre son.

```
sound("bd sd, hh*4")           // virgule = simultané  
sound("bd sd").bank("RolandTR909") // change tout le kit
```

bd grosse caisse · **sd** caisse claire · **rim** rimshot · **cp**
clap · **hh** charley fermé · **oh** ouvert · **lt** **mt** **ht** toms ·
rd ride · **cr** crash

Banques : **RolandTR909** **RolandTR808**
RolandTR707 **AkaiLinn** **RhythmAce** **CasioRZ1**
ViscoSpaceDrum **OberheimDMX**

RYTHMES EUCLIDIENS

Rythme euclidien = k coups répartis le plus régulièrement possible sur n pas. La répartition se calcule avec l'algorithme d'Euclide (celui du PGCD), et retombe sur de nombreux rythmes traditionnels.

```
$: sound("bd(3,8)")      // 3 sur 8  
$: sound("hh(7,16)")     // 7 sur 16  
$: sound("rim(5,8,2)")   // 5/8, décalé 2
```

(3,8) tresillo

(5,8) cinquillo

(7,16) charley nerveux

(9,16) dense, dub

(k , n , rotation) : la rotation décale le départ de r pas.

Lien avec les rythmes traditionnels : G. Toussaint, « *The Euclidean Algorithm Generates Traditional Musical Rhythms* », 2005.

CLÉ

Le son reste le même, seul le placement des coups change : modifier un seul nombre donne une nouvelle variante rythmique.

EMPIILER ET MUTER

`$:` en début de ligne = un pattern qui joue en même temps que les autres. Chaque `$:` est indépendant : on l'ajoute, on le modifie ou on le coupe seul.

```
$: sound("bd*4").bank("RolandTR909") // kick sur chaque temps  
$: sound("~ cp").bank("RolandTR909") // clap au 3e temps  
_$: sound("hh*8").gain(.5)           // underscore = muet
```

Le `_` est l'outil d'arrangement : si les lignes sont écrites à l'avance, muettes de base, on peut les allumer selon l'envie.

→ Permet de mixer des éléments pré-écrits en enlevant ou rajoutant des underscores.

CLÉ

Une ligne `$:` par élément musical. Deux éléments dans la même ligne ne peuvent pas être mutés séparément. ■

STACK() ET ALL()

```
stack(           // superpose
  sound("bd*4"), // kick
  sound("~ cp"), // clap
  sound("hh*8").gain(.5), // charley
).room(.2)       // reverb
```

Même résultat que trois `$:`, mais un effet peut être appliqué à l'ensemble d'un coup.

```
all(x => x.room(.4)) // reverb partout
all(x => x.lpf(300))  // tout étouffé
```

`all()` : FAQ Strudel.

`all(f)` applique `f` à chaque ligne `$:` du panneau. 2e ligne : un passe-bas à 300 Hz étouffe tout d'un coup, c'est le break ; commenter la ligne fait revenir le morceau.

`/!\`

`all()` n'est pas un pattern : `_$: all(...)` échoue. Pour le rendre commutable, commenter et décommenter la ligne.

PARTIE II

MATIÈRE SONORE

notes • accords • effets • enveloppes • signaux • transformations



DES HAUTEURS

Hauteur = note plus ou moins aiguë. Noms anglo-saxons : `c d e f g a b` = do ré mi fa sol la si.
Numéros MIDI : un demi-ton par unité, `48` = do3, `60` = do4.

```
// 4 notes jouées au piano
note("c e g b").sound("piano")
note("c2 eb3 g4") // nom + octave
note("48 52 55") // numéros MIDI
```

- ▶ lettres `c d e f g a b`, altérations `c# eb`
- ▶ octave collée : `c2` grave, `c5` aigu, défaut 3
- ▶ ou numéros MIDI
- ▶ sans `.sound()` : onde triangle

SOURCES SONORES

```
sawtooth square sine triangle . piano . toute la banque General MIDI : gm_epiano1
gm_acoustic_bass gm_harpsichord gm_accordion gm_banjo gm_trumpet gm_sitar
gm_marimba ...
```

JOUER JUSTE SANS SOLFÈGE

Degré = rang d'une note dans la gamme, en partant de 0 : 0 la tonique, 1 la note suivante... 7 la tonique une octave plus haut (gamme à 7 notes).

Avec un `scale()`, `n()` donne un **degré** et pas directement une note → pas de fausse note possible !

```
n("0 2 4 6")           // degrés de la gamme
  .scale("C:minor")      // do mineur
  .sound("piano")        // timbre
```

```
n("0 .. 7")            // degrés 0 à 7
  .scale("D:dorian")     // ré dorien
  .sound("sawtooth")     // dent de scie
```

EN JAM

On détermine la gamme commune au début, et tout le monde écrit la même chose dans `.scale()`, ce qui empêche de sonner faux.

LES GAMMES

Gamme = choix de quelques notes parmi les 12 de l'octave (souvent 5 ou 7), qui donne sa couleur à la mélodie. Écriture `tonique:type` : `C:minor` = do mineur, la tonique est la note de départ.

Un seul mot

`major` `minor` `dorian` `phrygian` `mixolydian`
`lydian` `chromatic`
`hirajoshi` `iwato` `in-sen` `kumoi` (Japon) · `pelog`
(Indonésie) · `persian` · `ritusen`

Plusieurs mots : des deux-points

`"C:major:pentatonic"` `"A:minor:pentatonic"`
`"C:major:blues"` `"D:harmonic:minor"`
`"E:phrygian:dominant"` `"D:todi:raga"` (Inde)

DEGRÉ PLUS GRAND QUE LA GAMME

La gamme recommence une octave plus haut. `C:major` (7 notes) : `7` = do aigu, `9` = mi aigu.

`C:major:pentatonic` (5 notes) : `5` = do aigu. Négatif : on descend, `-1` = si grave en `C:major`.

MANIP

Garder les mêmes degrés, faire varier la gamme seule :

`n("0 2 4 2").scale("<C:minor C:dorian F:major>")`. Une mélodie, trois couleurs.

ACCORDS : TROIS ÉCRITURES

Accord = plusieurs notes jouées en même temps. Chiffrage = son nom abrégé : **Cm7** do mineur septième, **G7** sol septième, **Ab^7** la bémol septième majeure.

```
// 1. virgules = simultan  
note("[c3,eb3,g3] [f3,ab3,c4]").sound("gm_epiano1") // 2 accords, notes empil  es

// 2. intervalles (0 fondamentale, 3 tierce mineure, 4 majeure, 7 quinte)
note("<[0,4,7] [0,3,7]>".add("<53 48>")).sound("piano") // + 53 = fa3, + 48 = do3

// 3. chiffrage : le placement des notes est automatique
chord("<Cm7 Fm7 Ab^7 G7>") // 4 accords, un par cycle
  .mode("above:c3").voicing() // plac  s au-dessus de do3
  .sound("gm_epiano1") // piano   lectrique
```

Forme 2 : worksheet Music x Code (Peckham Digital). Forme 3 : recettes de la doc Strudel.

CONTREPOINT

Contrepoint = plusieurs lignes mélodiques indépendantes jouées ensemble. Canon : une même mélodie reprise en décalé par une autre voix. Arpège : notes d'un accord jouées une par une.

```
// off() : copie décalée dans le temps, transposée
note("c3 eb3 g3").off(1/8, x => x.add(12)).sound("triangle") // copie 1/8 après, +12

// deux off empilés : mini-canon
"0".off(1/3, add(2)).off(1/2, add(4)) // 3 voix : 0, +2, +4, décalées
  .n().scale("C:minor")               // lues comme degrés de do mineur

// arp() réordonne les notes d'un accord
note("<[c,eb,g] [f,ab,c4]>").arp("0 1 2 1").sound("piano") // notes 0 1 2 1
```

Le double `off()` vient de la recette « Arpeggios » de la doc Strudel.

CLÉ

`off()` est un raccourci très efficace : une seule ligne fabrique une deuxième voix.

LES EFFETS

Effet = traitement du son une fois produit : volume, filtre, écho, réverbération. S'ajoute en fin de ligne, `.nom(valeur)`. Filtre = coupe une partie des fréquences : le passe-bas garde les graves.

`.gain(n)` volume

`.lpf(hz)` passe-bas, étouffe

`.hpf(hz)` passe-haut, dégraisse

`.room(n)` réverbération

`.delay(n)` écho

`.pan(n)` stéréo

`.speed(n)` vitesse de lecture, négatif = à l'envers

`.crush(n)` bit-crusher, 16 propre à 2 destructeur

`.distort("2:.4")` saturation

`.vowel("a e i")` filtre de voyelle

`.clip(n)` durée des notes

LES EFFETS EN PRATIQUE

```
$: sound("bd*4").bank("RolandTR909").gain(.9) // kick, volume 0.9
$: sound("hh*8").gain(".4 1 .4 1").hpf(6000).pan("<.3 .7>") // accents, aigus, pan
$: sound("~ cp").room(.5).rsize(3).delay(.3).delaytime(.375) // reverb, écho 0,375 s
```

/\

Reverb et écho ne sont pas propres à une ligne : toutes les lignes passent par le même bus d'effets (l'« orbit » 1) et se partagent une seule reverb et un seul écho. Deux `room()` différents se contredisent, deux reverbs à fond noient le morceau. Plafond raisonnable : `.5`. `.orbit(2)` envoie une ligne sur un bus à part.

CLÉ

Chaque paramètre accepte un pattern, pas seulement un nombre : `.lpf("400 2000")` ferme puis ouvre le filtre dans le cycle, `.gain(".4 1")` crée des accents.

ENVELOPPE DE VOLUME

Enveloppe = évolution d'un paramètre pendant la durée d'une note. Ici le volume : comment il monte, se tient, s'éteint.

```
note("c3 e3 f3 g3").sound("sawtooth")           // 4 notes, dent de scie  
  .attack(.01).decay(.15).sustain(0).release(.1) // percussif  
  
  .adsr(".01:.15:0:.1") // équivalent compact
```

`attack` montée · `decay` chute · `sustain` niveau tenu de 0 à 1 · `release` extinction.

ENVELOPPE DE FILTRE

Même principe appliqué au filtre : à chaque note, le passe-bas s'ouvre puis se referme. Plus `lpenv` est grand, plus l'ouverture monte haut.

```
note("c2 c2 eb2 g2").sound("sawtooth") // basse en dent de scie
    .lpf(300)                             // point de départ
    .lpenv(4)                             // amplitude de l'ouverture
    .lpa(.05)                             // temps d'ouverture
    .lpd(.12)                             // temps de fermeture
    .lpq(8)                               // résonance
    .ftype("24db")                         // pente : 12db, 24db, ladder
```

Adapté de la recette « Filter Envelopes » de la doc Strudel.

LA TECHNO RÉSUMÉE (MAL)

L'ouverture progressive du filtre fait l'essentiel du caractère acid et techno. ■

SIGNAUX

Signal = valeur qui varie en continu dans le temps, une courbe plutôt qu'une suite de valeurs écrites.
Branché sur un paramètre, il le fait bouger tout seul.

```
// passe-bas ondulant, 400 à 6000 Hz, 8 cycles  
sound("hh*16").lpf(sine.range(400, 6000).slow(8))  
// volume au hasard, en douceur, de 0.7 à 1  
sound("bd*4").gain(perlin.range(.7, 1))  
// hauteur en rampe de 36 à 48, 8 paliers  
note(saw.range(36, 48).segment(8)).s("sawtooth")
```

`sine` `saw` `tri` `square` oscillations

`rand` aléatoire brut · `perlin` aléatoire lissé ·

`irand(n)` entiers

`.range(a,b)` recadre, `.rangex()` en courbe

`.slow(n)` ralentit la courbe

`.segment(n)` n valeurs par cycle

REMÈDE ANTI RÉPÉTITION ROBOTIQUE

`perlin` sur `.gain()`, `.pan()` ou `.lpf()`. L'aléatoire lissé excellent pour avoir des patterns plus vivants. ■

TRANSFORMER : LE TEMPS

Transformer = appliquer une fonction à un pattern déjà écrit, sans retoucher sa mini-notation. Ici, celles qui agissent sur le temps : vitesse, sens, décalage, rythme.

```
.fast(2) .slow(2) // accepte "<1 2>"  
.rev()           // à l'envers  
.palindrome()    // aller-retour  
.hurry(2)        // vitesse + hauteur  
.ply(2)          // double chaque son
```

```
.iter(4)          // décale le départ  
.late(.02)        // micro-décalage  
.swingBy(1/3, 8)  // shuffle  
.struct("x ~ x x") // impose un rythme  
.mask("<1 0>")     // ouvre et ferme
```

CLÉ

Ces fonctions ne modifient pas des notes : elles modifient le pattern entier, et se cumulent.

TRANSFORMER : LE CONDITIONNEL

La transformation ne s'applique que de temps en temps : certains cycles, ou au hasard. `f` = la transformation, écrite `x => x.speed(2)` : « le pattern x devient x avec `.speed(2)` ».

```
.every(4, rev)           // un cycle sur quatre
.lastOf(8, x => x.fast(2)) // le dernier cycle sur huit
.sometimes(x => x.speed(2)) // 50 % des événements
.sometimesBy(.25, x => x.crush(4)) // 25 %
.often(f) .rarely(f)      // 75, 25 %
.almostNever(f) .almostAlways(f) // 10, 90 %
.degradeBy(.3)           // en retire 30 %
.someCycles(x => x.hpf(2000)) // par cycle entier
```

TRANSFORMER : COPIES ET STÉRÉO

Ces fonctions gardent le pattern d'origine et lui ajoutent une copie transformée : à droite dans la stéréo (`jux`), en même temps (`superimpose`) ou plus tard (`off`). Stéréo = son réparti entre gauche et droite.

```
.jux(rev) // gauche normal, droite inversée
.juxBy(.5, rev) // idem, stéréo à moitié
.superimpose(x => x.add(12).gain(.4)) // copie à l'octave
.off(.125, x => x.hpf(3000).gain(.5)) // copie décalée d'1/8
```

MANIP

`.jux(f)` joue le pattern tel quel à gauche et une copie transformée par `f` à droite. Avec `rev`, la droite joue le motif à l'envers : effet stéréo immédiat, sur n'importe quel pattern.

COMBO DE RÉFÉRENCE POUR SYNTHÉTISER

```
n("0 3 5 7").scale("C:minor").sound("sawtooth") // 4 degrés de do mineur
.lpf(500).lpenv(3).lpq(6) // filtre à enveloppe
.off(1/8, x => x.add(12).gain(.4)) // écho à l'octave
.sometimesBy(.2, x => x.ply(2)) // 20 % doublées
.every(4, rev) // à l'envers, 1 cycle sur 4
.room(.3) // reverb
```

Une ligne qui ne se répète jamais tout à fait.

DÉCOUPER UNE BOUCLE

Break = boucle de batterie enregistrée, ici l'« amen break » (The Winstons, 1969). Découpé en tranches rejouées dans un autre ordre : base de la jungle.

```
samples('github:yaxu/clean-breaks') // charge les breaks depuis GitHub

$: s("amen/4").fit().chop(16).cut(1)      // break sur 4 cycles, 16 tranches
  .sometimesBy(.5, ply("2"))             // 50 % doublées
  .sometimesBy(.25, mul(speed("-1")))    // 25 % à l'envers

$: s("amen/4").fit()                     // break sur 4 cycles
  .slice(8, "<0 1 2 3 [4 4] 5 6 7>*2")    // 8 tranches, ordre imposé
  .cut(1)                                // sans chevauchement
```

Recette « Chopping Breaks » de la doc Strudel.

⚠

Sans `.cut(1)`, les tranches se superposent et le break devient une bouillie. Attention sur les samples longs.

LES OUTILS DE DÉCOUPE

Échantillon long (boucle, break, voix) : sa durée ne suit pas le cycle. Le **caler** sur le tempo, puis le **découper** en tranches égales, rejouées dans l'ordre ou non.

CALER

`.fit()` étiré ou compressé pour durer exactement son événement

`.loopAt(2)` idem, sur 2 cycles

`.begin(.25)` démarre au quart du fichier ; `.end()` fixe la fin

DÉCOUPER

`.chop(16)` 16 tranches dans l'ordre, transformables une à une

`.slice(8, "0 3 1")` 8 tranches (0 à 7), joue celles indiquées dans cet ordre

`.splice(8, "...")` comme `slice`, tranches étirées à la taille du pas

`.cut(1)` une tranche coupe la précédente

MANIP

Ne changer que l'ordre des tranches dans `.slice()` : déjà un breakbeat original.

Plus loin : `samples({ nom: 'https://...mp3' })` charge un fichier en ligne, `.striate(n)` entrelace plusieurs échantillons.

PARTIE III

CONSTRUIRE UN MORCEAU

une écriture, plusieurs chemins • entrer • arrange • pick



UN RÉSULTAT, SEPT ÉCRITURES

```
$: s("bd ~ bd ~")           // silences écrits
$: s("bd*2")                 // multiplication
$: s("bd").fast(2)           // motif accéléré
$: s("bd(2,4)")              // euclide : 2 coups sur 4 pas
$: s("bd*4").struct("x ~ x ~") // rythme imposé
$: s("bd*4").mask("1 0 1 0") // 2 pas sur 4 retirés
$: s("bd*4").gain("1 0 1 0") // 2 pas sur 4 à volume nul
```

Même rythme, poignées différentes. `bd*2` se module (`"bd*<2 4>"`), `gain` se dose (`"1 0 .5 0"`), `mask` se pilote par cycle (`"<1 0>"`). Avec `gain(0)` l'événement existe encore, muet ; avec `mask` il disparaît.

FAIRE ENTRER UN ÉLÉMENT

```
_$: s("hh*8") // 1. mute : le _ retiré à la main

$: s("hh*8").mask("<0!4 1!4>") // 2. masque : 4 cycles muets, 4 joués

$: s("hh*8").gain("<0 .3 .6 1>") // 3. volume : monte par paliers
$: s("hh*8").gain(saw.range(0, 1).slow(8)) // ou en fondu continu, 8 cycles

$: s("hh*8").fast("<.5 1 2>") // 4. densité : 4, 8 puis 16 coups
$: s("hh*<4 8 16>") // même effet, en mini-notation
```

CLÉ

Le mute se joue à la main, en direct. `mask`, `gain` et `fast` s'écrivent d'avance et font évoluer l'élément seuls, cycle après cycle.

ARRANGE() : LA FORME ÉCRITE D'AVANCE

Section = passage du morceau (intro, refrain, break). `let nom = ...` donne un nom à un pattern pour le réutiliser plus bas.

```
let intro = s("bd*4, hh*8")           // section A : batterie seule
let plein = stack(                     // section B, deux couches
  s("bd*4, hh*8, ~ cp"),               // batterie complète
  note("<c2 eb2>*4").s("sawtooth").lpf(600), // basse
)

$: arrange(                           // enchaîne les sections, puis boucle
  [4, intro],                         // 4 cycles d'intro
  [8, plein],                         // 8 cycles pleins
  [2, plein.lpf(400)],                // 2 cycles étouffés
)
```

`/!\`

La forme boucle : 14 cycles, puis retour à l'intro. `arrange()` ne gouverne que sa ligne `$:`, le reste tourne librement.

PICK() : DES SECTIONS À LA LETTRE

`{ a: ..., b: ... }` = un objet : des patterns rangés sous des étiquettes. `pick()` va chercher le pattern de l'étiquette lue dans la chaîne.

```
let sections = {                // sections nommées par une lettre
  a: s("bd*4, hh*8"),           // a : batterie
  b: s("bd*4, hh*8, ~ cp"),      // b : avec clap
  c: s("bd ~ ~ ~").room(.6),     // c : respiration
}

$: "<a a b b a b c c>".pick(sections) // une lettre par cycle
```

L'ordre des sections tient dans une chaîne : `"<a a b b a b c c>"`. Changer une lettre change la forme, sans toucher aux sections.

LES GESTES D'UN MORCEAU

faire entrer, sortir	<code>_\$. .mask("<0 1>") . .gain("<0 1>") . arrange()</code>
densifier	<code>.fast("<1 2>") . "hh*<4 8 16>" . .ply("<1 2>") . .degradeBy("<.6 .3 0>")</code>
faire monter	<code>.lpf(saw.range(300, 4000).slow(16)) . .gain(saw.slow(8)) . .room("<0 .3 .6>")</code>
finir une phrase	<code>.lastOf(4, x => x.ply(2)) . .lastOf(8, rev) . "<a a a b>"</code>
casser	<code>all(x => x.lpf(300))</code> · tout muter sauf un élément
changer de section	<code>arrange()</code> · <code>pick()</code> · un <code>"<...>"</code> lent sur la gamme ou les accords

CLÉ

Chaque ligne = un geste, et plusieurs écritures pour l'obtenir, au résultat voisin. En retenir une par geste suffit pour commencer ; les autres servent à varier quand la première devient prévisible à l'oreille. ■

PARTIE IV

IMAGE

visus strudel • hydra solo • hydra dans flock



VISUS STRUDEL

Visu = image des événements joués, dessinée en direct par Strudel à partir du pattern : chaque note est représentée par un trait ou un point.

Écriture	Effet	flok
<code>.punchcard()</code>	grille de points derrière le code	non
<code>.pianoroll()</code>	idem, avant les transformations suivantes	oui
<code>.scope()</code>	oscilloscope	oui
<code>.spectrum()</code>	analyseur de spectre	non
<code>all(pianoroll)</code>	une visu pour tous les patterns, ligne à part	oui
<code>._punchcard()</code>	la même, à l'intérieur du code	non
<code>._pianoroll()</code>	pianoroll en ligne, dans le code	oui
<code>._spiral()</code> <code>._scope()</code> <code>._pitchwheel()</code>	autres visus en ligne, empilables	?
<code>slider(.5, 0, 1)</code>	curseur cliquable dans le code	non

VISUS : LE CODE

```
note("c2 a2 eb2").euclid(5,8).s("sawtooth")    // basse, 5 coups sur 8
    .lpenv(4).lpf(300)._spiral({ steady: .96 }) // filtre, spirale sous le code

$: n("0 2 4").scale("C:minor").s("sawtooth") // 3 degrés de do mineur
    .lpf(slider(800, 100, 4000))              // curseur : 100 à 4000 Hz
```

Exemple `_spiral` : page « Visual Feedback » de la doc Strudel.

HORS FLOK

`slider()` est une extension de l'éditeur strudel.cc, absente de flok (demande ouverte : issue flok 361, octobre 2025).

Testé sur flok : `.scope()` `.pianoroll()` `._pianoroll()` oui, `.punchcard()` `._punchcard()` `.spectrum()` non. Un seul affichage : le dernier bloc actualisé l'impose.

Remplacement d'un slider en jam : un pattern `"<800 1200 2000>"`, ou la valeur réécrite à la main.

HYDRA DANS STRUDEL.CC

Hydra = synthétiseur vidéo : une source (oscillateur, bruit, forme) passe dans une chaîne de transformations, comme un son dans des effets. `.out()` affiche le résultat.

```
await initHydra() // toujours en première ligne

osc(8, 0.1, 0.6) // oscillateur : fréquence, vitesse, couleur
  .color(1, 0.2, 0.6) // teinte rouge, vert, bleu
  .kaleid(5) // kaléidoscope à 5 faces
  .rotate(() => time * 0.05) // rotation lente continue
  .modulate(noise(2, 0.1), 0.3) // déformation par un bruit
  .out() // envoi à l'écran

$: n("0 2 4 6").scale("C:minor").s("sawtooth").lpf(600) // son joué en parallèle
```

HYDRA : LES BRIQUES

sources `osc()` `noise()` `voronoi()` `shape()` `gradient()` `solid()` `src(o0)`

géométrie `.rotate()` `.scale()` `.kaleid()` `.pixelate()` `.repeat()` `.scrollX()`

couleur `.color()` `.colorama()` `.contrast()` `.saturate()` `.invert()` `.luma()`

mélange `.blend()` `.diff()` `.mult()` `.add()` `.layer()` `.mask()`

modulation `.modulate()` `.modulateScale()` `.modulateRotate()`

sortie `.out(o0)`, buffers `o0` à `o3`, `render()` les affiche en grille

CLÉ

Tout paramètre accepte une fonction du temps : `.rotate(() => time * 0.05)`.

Rôle et paramètres de chaque fonction : antisèches Hydra, partie Références.

BRANCHER LE SON SUR L'IMAGE

FFT = analyse du son par bandes de fréquences, des graves (bande 0) aux aigus. Chaque bande donne un nombre qui suit le volume dans cette zone.

```
// 1. H() : un pattern Strudel pilote un paramètre Hydra
let p = "<3 5 8 4>"           // motif commun au son et à l'image
shape(H(p)).kaleid(4).out()    // nombre de côtés = motif
$: n(p).scale("A:minor").piano() // même motif, en notes

// 2. FFT : l'image réagit au spectre
await initHydra({ detectAudio: true })           // active l'analyse du son
osc(10, 0.1).scrollY(() => a.fft[0] * 0.3).out() // défilement suit les graves
```

/\

`detectAudio: true` ouvre un second contexte audio : sur certaines machines le son de Strudel se coupe. Repli : `initHydra()` simple, et pilotage par `H()`.

HYDRA DANS FLOK

```
// panneau STRUDEL : nommer l'analyseur de l'instrument
$: note("c2 ~ eb2 ~").s("sawtooth").analyze("basse") // analyseur "basse"

// panneau HYDRA, pas un panneau Strudel
osc(6, 0.1, 0.8) // oscillateur
  .color(0.2, 1, 0.8) // teinte cyan
  .rotate(() => time * 0.03) // rotation lente
  .scale(() => 1 + fft(0, 8, { analyzerId: "basse" }) * 0.6) // zoom : basse
  .modulate(noise(3), 0.3) // déformation
  .out() // envoi à l'écran

useStrudelCanvas(s0) // canvas Strudel en source vidéo
src(s0).kaleid(4).out() // source passée au kaléidoscope
```

HYDRA DANS FLOK : LES RÈGLES

- ▶ `fft(0, 8)` sans option : tout le mix. Testé : ne réagit qu'avec un `.pianoroll()` actif.
- ▶ `.analyze("basse")` + `fft(0, 8, { analyzerId: "basse" })` : l'image suit ce seul instrument. Même nom possible sur plusieurs panneaux.
- ▶ `s0` = le canvas Strudel de la session, sans choix de panneau : dernier `.pianoroll()` actualisé. Un `.scope()` l'efface ; sans `.pianoroll()`, image noire.
- ▶ Pas de surlignage de mini-notation dans les panneaux Hydra.
`fft()`, `useStrudelCanvas()` et l'actualisation manuelle : *documentation flok*.

/\

Après actualisation d'un code Hydra qui demande la FFT, **actualiser à nouveau chaque panneau Strudel** (`Alt+Entrée`). Ce n'est pas automatique.

MANIP

Objectif minimal pour le poste visuel, une basse nommée `.analyze("basse")`, puis un seul nombre modifié par section :

```
osc(20, 0.05, 0.9).kaleid(() => 3 + fft(0, 8, { analyzerId: "basse" }) * 5).out()
```

PARTIE V

COLLECTIF

strudel.cc contre flock • conventions de session • arrangement par mutes



STRUDEL.CC CONTRE FLOK

RÉSERVÉ À STRUDEL.CC

- ▶ `._punchcard()` ; non testés : `._spiral()`
`._scope()` `._pitchwheel()`
- ▶ `._punchcard()` et `._spectrum()`, même sans underscore
- ▶ `slider()` et tout widget cliquable
- ▶ panneau de documentation, recherche de fonctions
- ▶ historique et patterns sauvegardés

IDENTIQUE DES DEUX CÔTÉS

- ▶ toute la syntaxe des patterns, sans exception
- ▶ `samples()` et les banques distantes GitHub
- ▶ surlignage de la mini-notation à la lecture
- ▶ `$:` `_$:` `all()`, MIDI
- ▶ `._scope()` `._pianoroll()` `._pianoroll()`,
un seul affichage à la fois

MÉTHODE D'ATELIER

Chercher sur strudel.cc, où les visus en ligne et les sliders servent à explorer. Jouer sur [flok](https://flok.cc), où seule la syntaxe compte. ■

CONVENTIONS UTILES POUR UNE SESSION

```
// ---- PAULO • BASSE -----  
// KEY : C minor
```

- ▶ Un panneau = une personne, ou un rôle. La gamme commune est écrite en commentaire et reprise par tout le monde dans `.scale()`.
- ▶ Une ligne `$:` par élément : une pour le kick, une pour les charleys, une pour la basse. Deux éléments dans la même ligne ne peuvent plus être mutés séparément.

LE TEMPO APPARTIENT À UN SEUL PANNEAU

`setcpm()` est écrit une fois, dans un seul panneau de la session. Si un deuxième est écrit ailleurs, les deux tempos vont se disputer l'horloge. ■

ARRANGEMENT PAR MUTES

```
// ---- SARAH • HARMONIE -----  
// KEY : C minor  
  
_$: chord("<Cm9 Ab^9>").mode("above:c3").voicing() // Cm9 puis Ab^9  
    .s("gm_epiano1").clip(.8).room(.5).gain(.5) // nappe de piano  
  
_$: n("0 2 4").scale("C:minor").s("sawtooth").struct("~ x ~ x") // 2 coups par cycle  
    .decay(.2).sustain(0).delay(.4).gain(.45) // stabs courts, écho  
  
// all(x => x.lpf(300)) break : décommenter
```

On peut écrire des lignes « à l'avance » les tester dans son coin, puis les ajouter muettes, pour les rendre entendables au moment souhaité en enlevant `_`.

PARTIE VI

DIX-HUIT EXEMPLES

neuf électroniques • neuf acoustiques



LE CATALOGUE

ÉLECTRONIQUE

techno · deep house · acid · jungle · boom bap · trap
808 · ambient · synthwave · polyrythmie

ACOUSTIQUE

jazz · bossa nova · valse musette · baroque · reggae
roots · percussions indiennes · bluegrass · gamelan ·
fanfare balkanique

À TESTER

Coller un bloc, actualiser, puis **rendre des lignes muettes** pour mieux sentir ce que fait le morceau – au delà des commentaires systématiques par ligne.

RÉPERTOIRE ÉLECTRONIQUE

batteries programmées • synthèse • boucles découplées



TECHNO HYPNOTIQUE

130 BPM 4/4 martelé · une seule note de basse qui s'ouvre lentement · stab en écho pointé

```
setcpm(130/4) // 130 BPM, 4 temps par cycle

$: s("bd*4").bank("RolandTR909").gain(.95) // kick 909 sur chaque temps
$: s("~ cp").bank("RolandTR909").gain(.7).room(.3) // clap au 3e temps
$: s("hh*8").bank("RolandTR909").gain(.3 .6).hpf(7000) // croches, accents, passe-haut
  .sometimesBy(.15, x => x.speed(2)) // 15 % une octave au-dessus
$: s("oh(3,8,2)").bank("RolandTR909").gain(.35).hpf(4000) // charley ouvert, 3 sur 8 décalé

$: note("<c1 c1 eb1 g1>*2").s("sawtooth") // basse, 2 notes par cycle
  .lpf(sine.range(200, 1400).slow(16)).lpq(9) // filtre ondulant, 16 cycles
  .lpenv(3).lpa(.02).lpd(.12) // coup de filtre à chaque note
  .distort("1.4:.5").gain(.8) // saturation

$: note("[c3,eb3,g3]").struct("~ x ~ x").s("sawtooth") // do mineur aux temps 2 et 4
  .attack(.005).decay(.12).sustain(0).hpf(300) // son sec, graves coupés
  .delay(.4).delaytime(.1875).delayfeedback(.6) // écho court, réinjecté
  .room(.4).gain(.5) // reverb
```

FAIRE JOUJOU la durée du balayage, `.slow(16)` vers `.slow(32)`, et `.lpq()` entre 4 et 20.

DEEP HOUSE

124 BPM charley ouvert en croches · accords à neuvièmes · basse courte et ronde

```
setcpm(124/4) // 124 BPM, 4 temps par cycle

$: s("bd*4").bank("RolandTR909").gain(.9) // kick sur chaque temps
$: s("~ cp").bank("RolandTR707").gain(.5).room(.25) // clap 707 au 3e temps
$: s("[~ oh]*2").bank("RolandTR909").gain(.35) // charley ouvert à contretemps
$: s("hh*16").gain(perlin.range(.15, .4)).hpf(8000) // doubles-croches, volume mouvant

$: n("0 ~ 2 ~ 3 ~ 2 ~").scale("F:minor").s("sawtooth") // basse en fa mineur
  .lpf(700).lpq(4).decay(.25).sustain(0).gain(.8) // filtrée, courte

$: chord("<Fm9 Bbm9 Ab^9 Cm9>") // grille de 4 accords
  .mode("above:c3").voicing() // placés au-dessus de do3
  .s("gm_epiano1").clip(.4).room(.5).gain(.55) // piano électrique, court
  .off(1/8, x => x.gain(.25).hpf(1200)) // écho 1/8 plus tard, aigu
```

FAIRE JOUJOU

`.off(1/8, ...)` vers `.off(1/16, ...)`, et la suite d'accords, qui porte tout le morceau.

ACID

128 BPM une 303 et rien d'autre · résonance haute · le mouvement du filtre compte plus que la mélodie

```
setcpm(128/4) // 128 BPM, 4 temps par cycle

$: s("bd*4").bank("RolandTR909").gain(.95) // kick sur chaque temps
$: s("~ sd").bank("RolandTR808").gain(.6) // caisse claire au 3e temps
$: s("[~ hh]*4").bank("RolandTR808").gain(.4) // charley à contretemps

$: note("[<c2 f1>/2](<3 5>,8)").s("sawtooth") // do2 puis fa1, 3 ou 5 coups sur 8
  .clip(perlin.range(.2, 1.2)) // durée des notes mouvante
  .lpf(sine.range(300, 1800).slow(12)) // filtre ondulant, 12 cycles
  .lpq(cosine.range(6, 16).slow(5)) // résonance ondulante, 5 cycles
  .lpenv(4).lpa(.02).lpd(.18).ftype("24db") // coup de filtre, pente 24 dB
  .rarely(x => x.add(note(12))) // 25 % à l'octave
  .distort("1.2:.5").room(.2) // saturation, reverb
```

Ligne acide adaptée de « acidic tooth » de eddyflux, motif d'exemple livré avec Strudel ([website/src/examples.mjs](https://github.com/eddyflux/strudel/blob/master/src/examples.mjs)).

FAIRE JOUJOU (`<3 5>,8`) vers (`<3 5 7>,16`) . `.lpq(20)` pour que ça pique.

JUNGLE

172 BPM break amen découpé · sub sinusoïdal très grave · accords lointains · kick irrégulier

```
setcpm(172/4)           // 172 BPM, 4 temps par cycle
samples('github:yaxu/clean-breaks') // charge les breaks depuis GitHub

$: s("amen/4").fit().chop(16).cut(1).gain(.9) // break sur 4 cycles, 16 tranches
  .sometimesBy(.4, ply("2"))                // 40 % doublées
  .sometimesBy(.2, mul(speed("-1")))         // 20 % à l'envers

$: s("bd*2").bank("RolandTR909").gain(.7).lpf(120) // kick sourd, 2 par cycle

$: note("<g1 g1 bb1 f1>").s("sine").clip(.8)      // sub, une note par cycle
  .attack(.01).release(.2).lpf(180).gain(.9)    // attaque nette, graves seuls

$: n("0 ~ ~ 3 ~ 5 ~ ~").scale("G:minor").s("gm_epiano1") // 3 notes de sol mineur
  .clip(.3).room(.6).delay(.4).delaytime(.28).gain(.4) // courtes, reverb, écho
```

Découpe du break : recette « Chopping Breaks » de la doc Strudel.

FAIRE JOUJOU remplacer `.chop(16)` par `.slice(8, "<0 1 2 3 [4 4] 5 6 7>*2")` et réécrire l'ordre.

BOOM BAP

86 BPM caisse claire lourde sur 2 et 4 · swing · grésillement · accords légèrement en retard

```
setcpm(86/4) // 86 BPM, 4 temps par cycle

$: s("bd ~ [~ bd] ~").bank("AkaiLinn").gain(.95) // kick : 1er temps, fin du 3e
$: s("~ sd ~ sd").bank("AkaiLinn").gain(.8).room(.2) // caisse claire, temps 2 et 4
$: s("hh*8").gain(perlin.range(.2, .45)).swingBy(1/3, 8) // croches swinguées
$: s("crackle*2").density(.4).gain(.25).hpf(1500) // craquements de vinyle

$: note("<c1 ~ f1 ~>").s("sine").lpf(150) // sub, une note tous les 2 cycles
  .decay(.5).sustain(.3).gain(.9) // enveloppe

$: chord("<Cm9 Fm9>").mode("above:c3").voicing() // Cm9 puis Fm9, sur do3
  .s("gm_epiano1").clip(.9).lpf(2200) // piano électrique étouffé
  .late(.02).room(.4).gain(.5) // léger retard, reverb
```

FAIRE JOUJOU

`.swingBy(1/3, 8)` vers `.swingBy(.2, 8)`. Le `.late()` sur les accords fait le laid-back.

TRAP 808

140 BPM, moitié-temps 808 qui glisse et sature · roulements de charleys · beaucoup de vide

```
setcpm(140/4) // 140 BPM, 4 temps par cycle

$: s("bd ~ ~ bd ~ ~ bd ~").bank("RolandTR808").gain(1) // kick 808 en 3 + 3 + 2
$: s("~ ~ sd ~").bank("RolandTR808").gain(.8) // caisse claire au 3e temps
$: s("hh*8").bank("RolandTR808").gain(.4) // charley en croches
  .sometimesBy(.3, x => x.ply("<2 3 4>")) // 30 % en rafales

$: note("<c1 c1 g0 ab0>").s("sine") // basse 808, une note par cycle
  .penv(-12).pdec(.12) // glissé de 12 demi-tons à l'attaque
  .decay(.9).sustain(0).lpf(200) // tenue longue, graves seuls
  .distort("1.5:.4").gain(1) // saturation

$: n("0 ~ 3 ~ 2 ~ ~ 5").scale("C:minor").s("triangle") // mélodie en do mineur
  .clip(.3).delay(.3).delaytime(.1875).room(.5).gain(.4) // courte, écho, reverb
```

FAIRE JOUJOU `.penv(-12)` est la glissade du 808 : essayer `-24`, et `.pdec(.3)`.

AMBIENT

1 cycle = 8 secondes aucun rythme · attaques longues · réverbération immense · l'aléatoire lisse remplace l'écriture

```
setcpm(60/8) // 60 BPM, 8 temps par cycle

$: note("d1").s("sine").attack(4).release(8).lpf(120).gain(.5) // bourdon, montée 4 s

$: n("<0 2 4 6>").scale("D:dorian").s("sawtooth") // une note de ré dorien par cycle
  .attack(2).release(4) // entrée et sortie lentes
  .lpf(sine.range(300, 1200).slow(16)) // filtre ondulant, 16 cycles
  .room(.9).rsize(8).pan(sine.slow(23)).gain(.35) // grande reverb, pan lent

$: n(irand(8).segment(2)).scale("D:dorian").s("triangle") // 2 notes au hasard par cycle
  .clip(.2).degradeBy(.6) // courtes, 60 % supprimées
  .delay(.6).delaytime(.75).delayfeedback(.7) // écho long, réinjecté
  .room(.8).gain(.3) // reverb

$: s("crackle").density(.6).hpf(2000).room(.5).gain(.2) // grésillement de fond
```

FAIRE JOUJOU

`setcpm(60/16)` double encore la lenteur. `.rsize()` jusqu'à 10.

SYNTHWAVE

110 BPM basse en croches régulières · lead carré à écho pointé · réverbération de gymnase

```
setcpm(110/4) // 110 BPM, 4 temps par cycle

$: s("bd ~ bd ~").bank("RolandTR707").gain(.9) // kick, temps 1 et 3
$: s("~ cp").bank("RolandTR707").room(.5).gain(.7) // clap au 3e temps
$: s("hh*8").bank("RolandTR707").gain(".3 .5") // croches accentuées

$: note("<a1 a1 f1 g1>*2").s("sawtooth") // basse, 2 notes par cycle
  .lpf(600).lpq(5).decay(.2).sustain(.1).gain(.7) // filtrée, courte

$: n("0 2 4 7 4 2".add("<0 5 3 -2>")).scale("A:minor").s("square") // arpège transposé
  .clip(.6).every(4, rev) // à l'envers, 1 cycle sur 4
  .delay(.35).delaytime(.1875).delayfeedback(.5) // écho
  .room(.4).gain(.45) // reverb
```

FAIRE JOUJOU

`.add("<0 5 3 -2>")` est la grille d'accords déguisée. `.s("square")` vers `.s("sawtooth")`.

POLYRYTHMIE

105 BPM trois euclidiennes qui ne tombent jamais ensemble · pas de grille 4/4

```
setcpm(105/4) // 105 BPM, 4 temps par cycle

$: s("bd(3,8)").gain(.9) // kick, 3 coups sur 8
$: s("rim(5,8,2)").gain(.6) // rimshot, 5 sur 8, décalé de 2
$: s("hh(7,16)").gain(.4).pan(sine.slow(7)) // charley 7 sur 16, pan lent
$: s("{lt mt ht, cp cp}").gain(.5).speed("<1 1.2>") // polymètre 3 contre 2

$: n("0 3 5 7".add("<0 2 -3>")).scale("E:phrygian").s("triangle") // mi phrygien, transposé
  .clip(.4).off(1/8, x => x.add(7).gain(.3)) // copie décalée, +7 degrés
  .room(.35).gain(.5) // reverb

$: note("<e1 e1 c1 b0>").s("sine").lpf(160) // sub, une note par cycle
  .decay(.4).sustain(.2).gain(.85) // enveloppe
```

FAIRE JOUJOU les trois couples euclidiens : (3,8) (5,8) (7,16) vers (5,12) (7,12) (9,16).

RÉPERTOIRE ACOUSTIQUE

instruments joués • métriques impaires • gammes non occidentales



JAZZ, TRIO SWING

130 BPM swingué ride qui porte le tempo · contrebasse qui marche · accords de septième

```
setcpm(130/4) // 130 BPM, 4 temps par cycle

$: s("rd*8").gain(".5 .3").swingBy(1/3, 4).room(.2) // ride en croches swinguées
$: s("~ sd").gain(.3).room(.25) // caisse claire discrète, 3e temps
$: s("bd ~ ~ [~ bd]").gain(.45).lpf(200) // grosse caisse feutrée

$: n("0 2 4 6 7 6 4 2").scale("F:major").s("gm_acoustic_bass") // walking bass, fa majeur
    .clip(.95).gain(.85).swingBy(1/3, 4) // notes liées, swing

$: chord("<Gm7 C7 F^7 Bb^7>") // grille II-V-I-IV
    .mode("above:c3").voicing() // placée au-dessus de do3
    .struct("~ x ~ [x ~]").swingBy(1/3, 4) // comping syncopé, swing
    .s("gm_piano").clip(.5).room(.3).gain(.5) // piano
```

FAIRE JOUJOU

`.swingBy(1/3, 4)` vers `.swingBy(.45, 4)` : le swing se creuse.

BOSSA NOVA

128 BPM guitare nylon syncopée · rimshot en clave · basse sur deux temps · rien ne dépasse

```
setcpm(128/4) // 128 BPM, 4 temps par cycle

$: s("rim(3,8,2)").gain(.35) // rimshot, 3 sur 8 décalé
$: s("hh*8").gain(perlin.range(.08, .2)) // croches très douces
$: s("bd ~ ~ bd ~ ~ ~ ~").gain(.5).lpf(180) // grosse caisse feutrée

$: n("0 ~ ~ 4 ~ 0 ~ ~").scale("A:minor").s("gm_acoustic_bass") // fondamentale et quinte
    .clip(.8).gain(.8) // notes presque liées

$: chord("<Am7 D7 G^7 C^7>") // grille de 4 accords
    .mode("above:c3").voicing() // placée au-dessus de do3
    .struct("~ x ~ x x ~ x ~") // rythme syncopé
    .s("gm_acoustic_guitar_nylon").clip(.4).room(.35).gain(.55) // guitare nylon
```

FAIRE JOUJOU le `struct()` de la guitare : c'est lui, et lui seul, qui fait la bossa. Déplacer un seul **X**.

VALE MUSETTE, À 3 TEMPS

186 BPM, 3 temps premier cycle hors du 4/4 · basse sur le temps, accords sur 2 et 3

```
setcpm(186/3) // 3 temps par cycle : 1 cycle = 1 mesure à 3/4

$: s("bd ~ ~").gain(.7) // grosse caisse, 1er temps
$: s("~ hh hh").gain(.3) // charley, temps 2 et 3

$: note("<c2 g1 c2 f1>").s("gm_acoustic_bass") // basse, 1 note par mesure
  .struct("x ~ ~").clip(.6).gain(.8) // sur le 1er temps

$: note("<[c3,eb3,g3] [b2,d3,g3] [c3,eb3,g3] [a2,c3,f3]>") // 4 accords, 1 par mesure
  .struct("~ x x").s("gm_accordion").clip(.3).gain(.5).room(.2) // accordéon, temps 2 et 3

$: n("<4 3 2 1 0 2 4 7>*2").scale("C:harmonic:minor") // mélodie, 2 notes par mesure
  .s("gm_accordion").clip(.8).gain(.45).vib(4) // accordéon, vibrato
```

FAIRE JOUJOU

`setcpm(bpm/3)` : tout le style tient dans ce diviseur. `/5` et `/7` ouvrent les métriques impaires.

BAROQUE, CLAVECIN

96 BPM une seule voix, décalée d'elle-même · clavecin sans nuances · basse en valeurs longues

```
setcpm(96/4) // 96 BPM, 4 temps par cycle

$: n("0 2 4 2 5 4 2 0".add("<0 -1 -3 -4>")).scale("D:minor") // motif en marche descendante
  .s("gm_harpsichord").clip(.9).gain(.55) // clavecin
  .off(1/4, x => x.add(-4).gain(.4)) // canon à la quinte inférieure

$: n("<0 -1 -3 -4>").scale("D:minor").s("gm_harpsichord") // basse, une note par cycle
  .clip(1.8).gain(.5) // notes tenues

$: n("0 2 4 7".add("<0 -1 -3 -4>")).scale("D:minor") // arpège, même marche
  .s("gm_harpsichord").fast(2) // clavecin, 2 fois plus vite
  .clip(.35).gain(.3).degradeBy(.15) // court, 15 % retirées
```

FAIRE JOUJOU le décalage du canon : $\frac{1}{4}$ vers $\frac{1}{8}$ ou $\frac{1}{3}$. La suite "<0 -1 -3 -4>" est la basse chiffrée.

REGGAE ROOTS

78 BPM grosse caisse sur le troisième temps, jamais sur le premier · orgue sur les contretemps

```
setcpm(78/4) // 78 BPM, 4 temps par cycle

$: s("~ ~ bd ~").gain(1).lpf(120) // one drop : kick au 3e temps seul
$: s("~ ~ sd ~").bank("RolandTR808").gain(.5).room(.3) // caisse claire avec le kick
$: s("[~ hh]*4").gain(.3) // charley à contretemps

$: n("0 ~ ~ 0 ~ 3 ~ ~").scale("A:minor").s("gm_acoustic_bass") // basse en la mineur
    .clip(.9).lpf(400).gain(.95) // ronde, graves

$: note("[a3,c4,e4]").struct("~ x ~ x ~ x ~ x") // skank : accord à contretemps
    .s("gm_drawbar_organ").clip(.15).gain(.45) // orgue, très court
    .delay(.4).delaytime(.28).delayfeedback(.55).room(.3) // écho dub, reverb
```

FAIRE JOUJOU monter `.delayfeedback()` vers `.8` sur le skank, et le couper d'un coup : c'est tout le dub.

PERCUSSIONS INDIENNES

100 BPM banque mridangam, syllabes de konnakol · bourdon continu

```
setcpm(100/4) // 100 BPM, 4 temps par cycle

$: sound("tha dhi thom nam").bank("mridangam").gain(.8) // 4 frappes de mridangam
$: sound("thom ~ ~ tha ~ nam ~ ~").bank("mridangam") // 2e motif
  .gain(.5).speed(1.2) // plus doux, plus aigu
$: sound("dhi*8").bank("mridangam").gain(.3).degradeBy(.4) // remplissage, 40 % retirées

$: note("<d2 d2 a1 d2>").s("sine").lpf(140).clip(1).gain(.7) // bourdon

$: n("0 1 2 4 5".add("<0 2>")).scale("D:todi:raga") // mélodie en raga todi
  .s("gm_sitar").clip(.5).room(.4).gain(.5) // sitar
```

Noms d'échantillons et banque mridangam : page « Stepwise » de la doc Strudel.

FAIRE JOUJOU remplacer "tha dhi thom nam" par un cycle de 5 ou 7 frappes. **D:phrygian:dominant** si la gamme todi est trop âpre.

BLUEGRASS

150 BPM roll de banjo en continu · contrebasse sur 1 et 3 · violon tenu par-dessus · pas de batterie

```
setcpm(150/4) // 150 BPM, 4 temps par cycle

$: n("<0 4>").scale("G:major").s("gm_acoustic_bass") // fondamentale puis quinte
  .struct("x ~ x ~").clip(.4).gain(.85)           // temps 1 et 3
$: s("~ rim").gain(.35)                             // rimshot au 3e temps

$: n("0 2 4 7 4 2".add("<0 3 4 0>")).scale("G:major:pentatonic") // motif transposé
  .s("gm_banjo").fast(2).clip(.3).gain(.5)           // banjo, 2 fois plus vite
  .sometimesBy(.2, x => x.add(7))                   // 20 % plus haut

$: n("<4 2 0 2>*2").scale("G:major").s("gm_fiddle") // violon, 2 notes par cycle
  .clip(1.2).gain(.4).vib(5).room(.3)               // liées, vibrato
```

FAIRE JOUJOU

`.add("<0 3 4 0>")` est la grille : trois accords, rien de plus. Le banjo supporte `.fast(3)`.

GAMELAN

90 BPM gamme pelog · deux motifs entrelacés en polymètre · gong grave tous les quatre temps

```
setcpm(90/4) // 90 BPM, 4 temps par cycle

$: note("c1").s("sine").struct("x ~ ~ ~").clip(2).lpf(90).gain(.8) // gong, 1er temps

$: n("0 2 4 2".add("<0 1>")).scale("C:pelog") // motif en pelog
  .s("gm_marimba").clip(.4).gain(.5) // marimba

$: n("2 0 4 2".add("<0 1>")).scale("C:pelog") // motif imbriqué
  .s("gm_kalimba").fast(2).clip(.3).gain(.4) // kalimba, 2 fois plus vite

$: n("{0 2 4, 1 3}").scale("C:pelog") // polymètre 3 contre 2
  .s("gm_steel_drums").clip(.3).gain(.35).room(.4) // steel drums

$: s("hh(5,16)").gain(.2) // charley, 5 sur 16
```

FAIRE JOUJOU **C:pelog** vers **C:hirajoshi** ou **C:iwato**. Le polymètre **{0 2 4, 1 3}** est le cœur de l'entrelacement.

FANFARE BALKANIQUE, À 7 TEMPS

160 BPM, 7 temps mesure à 7/8 groupée 3+2+2 · cuivres · gamme à seconde augmentée

```
setcpm(160/7) // 160 BPM, 7 temps par cycle

$: s("bd ~ ~ bd ~ sd ~").gain(.8) // découpe 3 + 2 + 2
$: s("hh*7").gain(".5 .3 .3 .5 .3 .5 .3") // 7 charleys, accents 3 + 2 + 2

$: note("<d2 a1 d2 g1>").s("gm_tuba") // tuba, une note par cycle
  .struct("x ~ ~ x ~ ~").clip(.5).gain(.8) // temps 1 et 4

$: n("0 1 2 4 2 1 0".add("<0 3>")).scale("D:phrygian:dominant") // mélodie, transposée
  .s("gm_trumpet").clip(.45).gain(.5).room(.3) // trompette
  .off(1/14, x => x.add(2).gain(.3)) // 2e voix, 1/2 temps après

$: note("[d3,f3,a3]").struct("~ x ~ ~ x ~ ~") // ré mineur, temps 2 et 5
  .s("gm_brass_section").clip(.2).gain(.4) // cuivres, très court
```

FAIRE JOUJOU le groupement `"bd ~ ~ bd ~ sd ~"` en 3+2+2. Essayer 2+2+3, puis `setcpm(160/5)`.

CROISER LES FAMILLES

Croiser deux bases donne souvent mieux qu'en choisir une.

Dans une famille

la batterie jungle sous les accords deep house · la basse acid à 86 BPM · l'ambient avec un `bd*4` dessous

Entre les deux familles

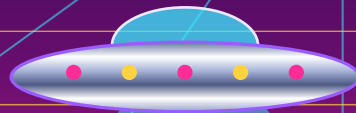
le banjo sur le kick techno · la grille de bossa en synthé carré · le mridangam à la place des charleys · la fanfare à 7 temps sur un sub

FAIRE JOUJOU Deux blocs dans deux panneaux flock, la moitié des lignes muettes. Testez des modifs pas à pas !

PARTIE VII

RÉFÉRENCES

antisèche • dépannage • plus loin • sources



ANTISÈCHE : LES BASES

MINI-NOTATION

<code>a b c</code>	séquence dans le cycle
<code>~</code>	silence
<code>[a b]</code>	sous-séquence
<code><a b></code>	un par cycle
<code>a*4</code> <code>a/2</code>	accélère, ralentit
<code>a,b</code>	simultané
<code>a!3</code>	répété
<code>a@3</code>	allongé
<code>a?</code> <code>a?0.2</code>	probabilité
<code>a:2</code>	numéro d'échantillon
<code>a(3,8,1)</code>	euclidien et rotation
<code>a b</code>	tirage par cycle
<code>{a b, c d e}</code>	polymètre
<code>0 .. 7</code>	suite de nombres

SESSION

<code>\$:</code>	pattern indépendant
<code>_\$:</code>	le même, muet
<code>all(f)</code>	agit sur tous les <code>\$:</code>
<code>hush()</code>	tout arrêter
<code>orbit(2)</code>	bus d'effets séparé
<code>setcpm(bpm/4)</code>	tempo en cycles par minute, si 4 temps par cycle

SONS ET HAUTEURS

<code>sound()</code> <code>s()</code>	échantillon ou onde
<code>bank()</code>	kit de batterie
<code>n()</code>	échantillon ou degré de gamme
<code>note()</code>	hauteur : <code>c3</code> , <code>48</code>
<code>scale()</code>	gamme : <code>"C:minor"</code>
<code>chord().voicing()</code>	accords chiffrés
<code>arp()</code>	arpège
<code>samples('github:...')</code>	charger une banque

ANTISÈCHE : TEMPS ET VARIATIONS

TEMPS

<code>fast()</code>	<code>slow()</code>	densité
<code>rev()</code>	<code>palindrome()</code>	sens de lecture
<code>iter(4)</code>		décale le départ
<code>ply(2)</code>		répète chaque événement
<code>hurry(2)</code>		accélère et transpose
<code>late()</code>	<code>early()</code>	micro-décalage
<code>swingBy(1/3, 8)</code>		shuffle
<code>struct("x ~ x")</code>		impose un rythme
<code>mask("<1 0>")</code>		ouvre et ferme

VARIATION

<code>every(4, f)</code>		un cycle sur quatre
<code>lastOf(8, f)</code>		le dernier cycle sur huit
<code>sometimesBy(.3, f)</code>		au hasard, dosé
<code>degradeBy(.3)</code>		retire des événements
<code>jux(rev)</code>		stéréo dissocié
<code>off(1/8, f)</code>		copie décalée
<code>superimpose(f)</code>		copie superposée
<code>chunk(4, f)</code>		par morceaux

SIGNAUX

<code>sine</code>	<code>saw</code>	<code>tri</code>	<code>square</code>	courbes
<code>rand</code>	<code>perlin</code>	<code>irand(n)</code>		aléatoire
<code>.range(a,b)</code>	<code>.rangex()</code>			recadrer
<code>.slow(n)</code>	<code>.segment(n)</code>			vitesse, discrétisation

ANTISÈCHE : LES EFFETS

<code>gain()</code>	volume	<code>room()</code> <code>rsize()</code>	réverbération
<code>lpf()</code> <code>hpf()</code> <code>lpq()</code>	filtres et résonance	<code>delay()</code> <code>delaytime()</code>	écho
<code>lpenv()</code> <code>lpa()</code> <code>lpd()</code>	enveloppe de filtre	<code>delayfeedback()</code>	répétitions de l'écho
<code>ftype()</code>	pente : 12db, 24db, ladder	<code>pan()</code> <code>speed()</code>	stéréo, vitesse de lecture
<code>attack</code> <code>decay</code> <code>sustain</code> <code>release</code>	enveloppe de volume	<code>crush()</code> <code>distort()</code>	saleté
<code>adsr(".01:.1:.5:.2")</code>	écriture compacte	<code>vowel()</code>	filtre de voyelle
<code>penv()</code> <code>pdec()</code>	enveloppe de hauteur	<code>clip()</code>	durée des notes

RAPPEL

Chaque paramètre accepte un pattern : `.lpf("400 2000")`, `room()` et `delay()` : un bus partagé par toutes les lignes, `.orbit(2)` en isole une. ■

ANTISÈCHE : SAMPLES ET HYDRA

SAMPLES LONGS

<code>fit()</code> <code>loopAt(2)</code>	caler sur la durée
<code>chop(16)</code>	tronçonner
<code>slice(8, "...")</code>	tronçonner et réordonner
<code>splice(8, "...")</code>	idem, tempo conservé
<code>cut(1)</code>	couper la tranche précédente
<code>begin()</code> <code>end()</code>	portion
<code>striate(n)</code>	entrelacer plusieurs échantillons

HYDRA

<code>await initHydra()</code>	première ligne, strudel.cc
<code>H(pattern)</code>	pattern vers paramètre Hydra
<code>a.fft[0]</code>	spectre, strudel.cc
<code>fft(0, 8)</code>	spectre, panneau flok
<code>.analyze("nom")</code>	analyseur d'un instrument, lu par <code>fft(0, 8, { analyzerId: "nom" })</code>
<code>useStrudelCanvas(s0)</code>	canvas Strudel en source
<code>.out(o0)</code> <code>render()</code>	sortie, grille de debug

ANTISÈCHE HYDRA : SOURCES ET GÉOMÉTRIE

SOURCES : PRODUISENT UNE IMAGE

<code>osc(freq, sync, offset)</code>	bandes qui défilent : nombre de bandes, vitesse, décalage des couleurs
<code>noise(scale, offset)</code>	bruit lisse en nuages : taille du grain, vitesse d'évolution
<code>voronoi(scale, speed)</code>	cellules façon mosaïque organique
<code>shape(sides, radius)</code>	polygone : 3 = triangle, 4 = carré, 100 ≈ cercle
<code>gradient(speed)</code>	dégradé de couleurs qui tourne
<code>solid(r, g, b)</code>	aplatissement d'une couleur, chaque valeur de 0 à 1
<code>src(o0)</code>	relit une sortie : base des effets de traînée

Paramètres : référence interactive hydra.ojack.xyz/api et documentation Hydra.

GÉOMÉTRIE : DÉPLACENT LES PIXELS

<code>.rotate(angle, speed)</code>	tourne l'image ; <code>speed</code> = rotation continue
<code>.scale(n)</code>	zoom : au-dessus de 1 grossit, en dessous rapetisse
<code>.kaleid(n)</code>	kaléidoscope à n faces
<code>.pixelate(x, y)</code>	gros pixels : x par y blocs
<code>.repeat(x, y)</code>	mosaïque : l'image répétée x fois par y
<code>.scrollX(n, speed)</code>	glisse vers la gauche ou la droite, en boucle ; <code>.scrollY()</code> en hauteur

ANTISÈCHE HYDRA : COULEUR, MÉLANGE, SORTIE

COULEUR

<code>.color(r, g, b)</code>	dose chaque canal : rouge, vert, bleu
<code>.colorama(n)</code>	fait tourner les teintes, effet psychédélique
<code>.contrast(n)</code> <code>.saturate(n)</code>	contraste, intensité des couleurs
<code>.invert()</code>	négatif
<code>.luma(seuil)</code>	ne garde que les zones plus claires que le seuil

SORTIE ET TEMPS

<code>.out(o0)</code>	envoie dans la sortie <code>o0</code> (affichée) ; <code>o1</code> à <code>o3</code> en réserve
<code>render()</code>	affiche les quatre sorties en grille
<code>() => time * .1</code>	paramètre recalculé à chaque image
<code>[2, 4, 8]</code>	liste : les valeurs alternent dans le temps

MÉLANGE : AVEC UNE 2E SOURCE

<code>.blend(src, n)</code>	fondue entre les deux images
<code>.add(src)</code> <code>.mult(src)</code>	addition (éclaircit), multiplication (assombrit)
<code>.diff(src)</code>	différence : couleurs inversées où les images se croisent
<code>.layer(src)</code>	pose la 2e image par-dessus, selon sa transparence
<code>.mask(src)</code>	n'affiche l'image que là où la 2e est claire

MODULATION : UNE IMAGE EN DÉFORME UNE AUTRE

<code>.modulate(src, n)</code>	décalle les pixels selon les couleurs de <code>src</code>
<code>.modulateScale(src, n)</code>	zoom variable selon <code>src</code>
<code>.modulateRotate(src, n)</code>	rotation variable selon <code>src</code>

QUAND ÇA NE MARCHE PAS

Symptôme	Cause habituelle
Aucun son	le navigateur attend un clic dans la page. Sinon : volume système, onglet muet.
Rien ne change	pattern non actualisé. <code>Ctrl+Entrée</code> , ou <code>Alt+Entrée</code> pour tout le panneau.
Erreur rouge en bas	presque toujours une parenthèse ou un guillemet. L'ancien pattern continue de jouer.
Un son manque	nom d'échantillon inconnu : Strudel ne joue rien et ne prévient pas.
Tout sature	trop de <code>room()</code> ou de <code>gain()</code> cumulés. Couper la reverb avant le volume.
Le break part en bouillie	<code>.cut(1)</code> manquant sur un sample tronçonné.
Hydra ne réagit pas au son, flok	actualiser à nouveau les panneaux Strudel après le code Hydra.
Le son se coupe avec Hydra, strudel.cc	<code>detectAudio: true</code> : conflit de contexte audio.
Deux tempos différents	quelqu'un a écrit un second <code>setcpm()</code> .

POUR ALLER PLUS LOIN

- ▶ `arrange([4, patternA], [2, patternB])` écrit une structure de morceau plutôt qu'une boucle.
- ▶ `pickRestart({a: ..., b: ...})` piloté par `"<a b a c>"` donne des sections nommées.
- ▶ Fonctions stepwise (`pace`, `stepcat`, `expand`, `grow`) : penser en nombre de pas plutôt qu'en cycles. Expérimental depuis la version 1.2.
- ▶ `.midi()` pilote du matériel externe, `.osc()` parle aux autres logiciels.
- ▶ Le REPL de strudel.cc embarque la référence complète des fonctions, avec exemples jouables.

ALLER PLUS LOIN : UNE FORME PAR INSTRUMENT

Une ligne `arrange()` par instrument : chacune dit quand son instrument joue, se tait ou change.

```
// 3 formes de même durée totale (16 cycles) : elles restent calées
$: arrange([8, s("bd*4")], [4, silence], [4, s("bd*4")]) // kick : trou au milieu
$: arrange([4, silence], [12, s("hh*8").gain(.4)]) // charley : entre au 4e
$: arrange([8, note("<c2 eb2>*4")], [8, note("<f2 g2>*4")]) // basse : 2 grilles
    .s("sawtooth").lpf(600) // timbre commun
```

cycles	0 à 3	4 à 7	8 à 11	12 à 15
kick	joue	joue	silence	joue
charley	silence	joue	joue	joue
basse	do, mi $\flat$	do, mi $\flat$	fa, sol	fa, sol

POURQUOI ÇA RESTE CALÉ

`arrange()` suit le compteur de cycles de Strudel, parti de 0 au lancement de la lecture, et non le moment où la ligne est actualisée : trois formes de 16 cycles repartent ensemble. Avec 16 et 12, décalage à chaque tour. `silence` = section vide.

ALLER PLUS LOIN : PICKRESTART()

```
let phrases = {                                // deux phrases nommées
  a: n("<0 2 4 6>").scale("C:minor"),           // a : 4 cycles, montante
  b: n("<7 5>").scale("C:minor"),               // b : 2 cycles, descendante
}

$: "<a@4 b@2>".pick(phrases).s("piano")         // a sur 4 cycles, b sur 2
$: "<a@4 b@2>".pickRestart(phrases).s("piano")  // idem, phrase relancée
```

`pick()` lit la phrase là où en est le temps global : au 2e tour (cycles 6 à 9), `a` joue `4 6 0 2`, elle démarre au milieu. `pickRestart()` la relance à sa première note : `0 2 4 6` à chaque tour.

MANIP

Muter l'une des deux lignes, écouter, puis l'autre. Changer `a@4` en `a@3` : la différence s'entend dès le premier tour. ■

`pickRestart()` et la notation `"<a@2 b@2>"` : page « Conditional Modifiers » de la doc Strudel.

ALLER PLUS LOIN : PENSER EN PAS

Pas = un élément de la mini-notation : `"bd sd cp"` compte 3 pas. Habituellement, une séquence remplit un cycle quelle que soit sa longueur. En stepwise, un pas garde la même durée partout.

```
$: s("bd*4") // repère : 4 temps
$: s("hh sd cp").pace(4) // 3 pas à 4 pas par cycle : se décale
$: stepcat("bd sd cp", "hh hh").s().pace(8) // 3 + 2 = 5 pas, 8 par cycle
$: s("tha dhi thom nam").bank("mridangam") // 4 frappes
  .grow("1") // 1 frappe, puis 2, 3, 4
$: s("tha dhi thom nam").bank("mridangam") // les mêmes
  .expand("3 2 1 1 2 3").pace(8) // pas étirés de 1 à 3 fois, 8 par cycle
```

/!

Fonctions expérimentales depuis la version 1.2 : noms et comportement peuvent changer. Une ligne à la fois, le reste muet.

Fonctions et exemples `grow`, `expand` : page « Stepwise » de la doc Strudel.

ALLER PLUS LOIN : QUI SUIVRE, PAS À PAS

Qui	Pourquoi	Où
Music x Code, Pattern Club	fiche débutant de Strudel : un concept par étape, rien d'implicite	strudel.patternclub.org/workshop/music-x-code
Lee Tusman	séance de cours Strudel (2025) : de <code>sound("bd")</code> aux breaks, jusqu'à un jeu à plusieurs ; licence CC	leetusman.com/dmsc_spring2025
Shelly Knotts	cours vidéo intermédiaire (1 h, 2024) : passer d'un beat reconnaissable au son abstrait	musichackspace.org , cours Strudel
Zach Krall	premiers pas Hydra (2022) : sources, modificateurs, sorties, dans cet ordre	zachkrall.hashnode.dev
Geikha	guides de la doc Hydra : JavaScript pour Hydra, patches génératifs, réaction au son expliquée	hydra.ojack.xyz/docs , Guides
Naoto Hieda	tutoriels Hydra en vidéo, dont une série pour débutants : montre aussi les tâtonnements	youtube.com/@naotohieda
strudel-tutorial, N. Gioannou	carnet de 32 notes (2026) : chaque opérateur décrit par son effet sur la structure du pattern, parcours d'une heure à « maîtrise » ; rédigé par une IA (Claude) qui n'entend pas le son	github.com/nikolasgioannou/strudel-tutorial
Officiel	Strudel Workshop, puis la doc Hydra « Getting Started » : exemples modifiables dans la page	strudel.cc/workshop • hydra.ojack.xyz/docs

ALLER PLUS LOIN : QUI SUIVRE, SOPHISTIQUÉ

Qui	Pourquoi	Où
Switch Angel	morceaux entiers codés de zéro en quelques minutes, sur Strudel dont elle est contributrice	youtube.com/@switch-angel
eddyflux	Felix Roos, créateur de Strudel : harmonies et rythmes bancals, morceaux d'exemple	youtube.com/@eddyflux
Kindohm	Mike Hodnick : dix ans de disques entièrement codés en TidalCycles, transposables à Strudel	kindohm.com · youtube.com/@kindohm
Dan Gorelick	violoncelliste, performances audiovisuelles codées, ateliers Strudel	dangorelick.com
Flor de Fuego	visuels Hydra en performance, caméras déformées, contributrice Hydra	flordefuego.xyz
Geikha	hyper-hydra : extensions Hydra (modes de fusion, géométrie), code lisible	github.com/geikha/hyper-hydra
Raphaël Forment	BuboBubo, Lyon et Paris : thèse sur les langages de live coding, auteur de Sardine, site en français, Cookie Collective	raphaelforment.fr · cookie.paris
À lire	« Live Coding: A User's Manual » (MIT Press, 2022, en libre accès) ; Roos et McLean, « Strudel: live coding patterns on the web » (ICLC 2023)	livecodingbook.toplap.org · iclc.toplap.org/2023

Recueils de code : strudel.cc/intro/showcase · hydra.ojack.xyz/garden · github.com/terryds/awesome-strudel.

LES LOGICIELS

Strudel	portage JavaScript de TidalCycles, par Felix Roos et Alex McLean puis la communauté. Version de référence : 1.2 « Kardinalschnitten », mai 2025. strudel.cc · codeberg.org/uzu/strudel
TidalCycles	l'ancêtre en Haskell, Alex McLean, 2009. Tout le modèle de patterns en vient. tidalcycles.org
Hydra	synthétiseur vidéo live-codable, Olivia Jack. hydra.ojack.xyz
flok	éditeur collaboratif P2P, Damián Silvani. flok.cc · codeberg.org/munshkr/flok

SOURCES UTILISÉES POUR CONSTRUIRE CES SLIDES

Source	Contenu repris
Strudel Workshop strudel.cc/workshop	La progression générale : sons courts avant hauteurs, hauteurs avant effets, effets avant transformations. L'idée du cycle, formulée là-bas « the longer the sequence, the faster it runs ». L'organisation des tableaux de l'antisèche.
Music x Code , worksheet Strudel Peckham Digital, 2025	Se limiter à une petite boîte à outils plutôt que tout retenir, et supprimer puis recommencer plutôt que déboguer. Le découpage sons courts, sons longs, effets, signaux, notes, accords.
« acidic tooth », eddyflux website/src/examples.mjs	La basse du style Acid en est adaptée : euclidienne sur deux notes, filtre balayé par <code>sine</code> , résonance par <code>cosine</code> , <code>clip()</code> piloté par <code>perlin</code> .
Strudel Recipes strudel.cc/recipes	Découpe du break amen, enveloppes de filtre, arpège par double <code>off()</code> , accords par <code>chord().voicing()</code> .
Visual Feedback et FAQ strudel.cc/learn	Distinction punchcard et pianoroll, rôle du préfixe <code>_</code> , exemple <code>._spiral()</code> . <code>all()</code> et la syntaxe <code>\$:</code> <code>_ \$:</code> .
Documentation flok munshkr.github.io/flok	Cibles, raccourcis, avertissement sur les serveurs publics, <code>fft()</code> et <code>analyzerId</code> , <code>useStrudelCanvas()</code> , actualisation manuelle. Issue 361 sur Codeberg (codeberg.org/munshkr/flok) pour les sliders.
Documentation Hydra hydra.ojack.xyz/docs	Vocabulaire des sources et transformations, principe de <code>a.fft[]</code> .

SOURCES, SUITE

Source	Contenu repris
Switch Angel relayé par Hackaday, 16 oct. 2025	Entrer par un style complet qui sonne immédiatement plutôt que par une démonstration de syntaxe.
awesome-strudel , terryds	Collection de morceaux complets, pour lire du code après l'atelier.
G. Toussaint , Bridges 2005	« The Euclidean Algorithm Generates Traditional Musical Rhythms » : rythmes euclidiens et musiques traditionnelles.
Typst typst.app	Langage de composition dans lequel ces slides sont écrites et compilées.
Claude , Anthropic via Claude Code	Assistance à la rédaction, à la structure, aux exemples de code et à la vérification des sources ; relu et corrigé à la main.

ALLEEEEEEEZZZ

